Machine Learning Intelligent Chip Design

指導教授: 陳坤志

HW3

Implement NoC by SystemC

(結報)

系級 中央大學電機四 B

學號 B132501

姓名 翁瑄妤

一、 模擬結果

主要在工作站上執行程式，使用 TA 提供的輸入資料（在 pattern 檔裡）進行測試。透過 SystemC 實作 4x4 Mesh-based NoC 模型，會顯示 "Congratulations!! Simulation completed at n th cycles"，表示所有 PE 成功交換並驗證 packet。

執行指令分為以下步驟:

(1) 打開所屬資料夾 (所需資料皆在 hw2 的資料夾裡)

cd hw3

(2) 執行 MakeFile (檔案由 TA 提供)

make

以下為執行結果:

```
03:33 mlchip074@ee25[~]% cd hw3
03:33 mlchip074@ee25[~/hw3]% make
g++ -I . -I /RAID2/COURSE/2025_Spring/mlchip/mlchipTA01/systemc-2.3.3/include -L . -L /RAID2/COURSE/2025_Spring/mlchip/mlchipTA01/systemc
-2.3.3/lib-linux64 -o run *.cpp -lsystemc-2.3.3 -Wl,-rpath,/RAID2/COURSE/2025_Spring/mlchip/mlchipTA01/systemc-2.3.3/lib-linux64
./run

        SystemC 2.3.3-Accellera --- Mar  4 2025 01:46:38
        Copyright (c) 1996-2018 by all Contributors,
        ALL RIGHTS RESERVED

  ============================
  +                          +        |\__|\
  +   Congratulations !!     +       / 0.0  |
  +                          +      /_____   |
  +   Simulation completed   +     /^ ^ ^ \  |
  +   at 228 th cycle        +    |^ ^ ^ ^ |w|
  +                          +     \m___m__|_|
  ============================
```

經執行後有出現 ”Congratulations” 的字樣。上傳資料至工作站之測試結果也皆為正確。

```
03:33 mlchip074@ee25[~/hw3/09_SUBMIT]% ./01_submit
[Info] Deadline check OK ...
[Info] File check OK ...
[Info] Copying TA's Makefile to your folder.
[Info] mlchip074 SystemC start
[Info] Checking source code for channels and interfaces...
[Info] Starting compilation...
[Info] Match Golden Result


Server_Account  mlchip074
Demo_Result     PASS
Error_Message   No_Error
Submiss_Date    2025/05/12
Submiss_Time    03:34:09
Sim_Time (s)    1.99


[Info] Your file will be submitted to:  TA folder
[Warning]  demo has been submitted.
[Warning]  It will overwrite your original file.
[Info] Now submit hw3_mlchip074.tar.gz file to system.
[Success] Copying Sucessfully.
=========================================================
                    Submit Report
=========================================================
Result          :   has been submitted.
Submission time :  2025/05/12 03:34:09
=========================================================
        ----------------------------
        --                        --       |\__||
        --  Congratulations !!    --      / 0.0  |
        --                        --    /_____   |
        --  Submission Sucessful!!--   /^ ^ ^ \  |
        --                        --  |^ ^ ^ ^ |w|
        ----------------------------   \m___m__|_|
   Please remember to check your submission with ./02_check !!
   Please remember to check your submission with ./02_check !!
   Please remember to check your submission with ./02_check !!
```

二、 模組說明 (未使用 Virtual Channel)

1. NI Design (core.h)

功能:

- 封裝 TA 提供的 PE
- 處理封包與 flit 之間的轉換（packet ⇌ flits）
- 執行 flit 的傳送與接收流程

重要變數：

- flit_rx / flit_tx：傳輸資料（sc_lv<34>）
- req_rx / req_tx：表示資料傳輸請求
- ack_rx / ack_tx：確認傳輸成功
- tx_buf：存放即將傳送的 flit 序列（由 packet 轉換而來）。
- tx_idx：目前正在傳送第幾個 flit 的 index。
- packet_requested：表示是否已向 PE 請求一個新的 packet 以避免重複請求。
- rx_buf：暫存接收到的 flits，直到組成完整的 packet 再傳給 PE check。

```
    sc_in  < sc_lv<34> > flit_rx;   // The input channel
    sc_in  < bool > req_rx;         // The request associated with the input channel
    sc_out < bool > ack_rx;         // The outgoing ack signal associated with the input channel
    // transmit
    sc_out < sc_lv<34> > flit_tx;   // The output channel
    sc_out < bool > req_tx;         // The request associated with the output channel
    sc_in  < bool > ack_tx;         // The outgoing ack signal associated with the output channel

    PE pe;
    int core_id;    // unique core id

    // internal transmission state
    std::vector<sc_lv<34> > tx_buf;     // buffer to send flit
    unsigned tx_idx;     // current index in buffer
    bool packet_requested;              // flag to indicate if a packet has been requeseted

    // internal receive state
    std::vector<sc_lv<34> > rx_buf;     // buffer to receive flits
```

子模組與函式:

- init_proc( ): 初始化 core_id，呼叫 pe.init(id)

```
void Core::init_proc() {
    pe.init(core_id);       // initialize PE and load input/output packets
    wait();         // wait for reset
}
```

- tx_proc( ): 從 PE 取得 packet 資料，轉換為 flits 並存入 tx_buffer。之後依據 ack_tx 信號逐筆傳送

```
void Core::tx_proc() {
    packet_requested = false;
    req_tx.write(false);
    flit_tx.write(0);
    wait();  // wait for reset

    // Stagger injection across cores
    for (int d = 0; d < core_id; ++d)
        wait();

    while (true) {
        wait();  // wait for next clock

        if (tx_buf.empty() && !packet_requested) {
            Packet* p = pe.get_packet();        // fetch next packet from PE
            if (p) {
                tx_buf = packet2flits(p);
                tx_idx = 0;
                packet_requested = true;
                delete p;
            }
        }

        if (!tx_buf.empty()) {
            flit_tx.write(tx_buf[tx_idx]);
            req_tx.write(true);
            // flit successfully sent
            if (ack_tx.read()) {
                tx_idx++;
                if (tx_idx >= tx_buf.size()) {
                    tx_buf.clear();
                    packet_requested = false;
                }
            }
        } else {
            req_tx.write(false);
        }
    }
}
```

- rx_proc( ): 接收 flit，並累積至 rx_buf，當收到 tail flit 即還原成 packet 並交給 PE check

```
void Core::rx_proc() {
    ack_rx.write(false);
    while (true) {
        wait();     // wait for next clock
        if (req_rx.read()) {
            sc_lv<34> flit = flit_rx.read();        // read incoming flit
            ack_rx.write(true);
            sc_lv<2> type = flit.range(33,32);

            // header flit
            if (type == "10") {
                rx_buf.clear();     // start of a new packet
            }
            rx_buf.push_back(flit);
            // tail flit (packet complete)
            if (type == "01") {
                Packet* p = flits2packet(rx_buf);
                pe.check_packet(p);     // verify correctness
                delete p;
                rx_buf.clear();
            }
        } else {
            ack_rx.write(false);
        }
    }
}
```

- packet2flits( ): 將封包轉為 flits

```cpp
std::vector<sc_lv<34> > Core::packet2flits(Packet* p) {
    std::vector<sc_lv<34> > v;
    // Construct header flit
    sc_lv<34> h = 0;
    h.range(33,32) = "10";                    // header type
    h.range(31,16) = sc_lv<4>(p->source_id);    // source id
    h.range(15,0) = sc_lv<4>(p->dest_id);      //destination id
    v.push_back(h);

    // Construct body/tail flits
    for (size_t i = 0; i < p->datas.size(); ++i) {
        float f = p->datas[i];
        unsigned u;
        std::memcpy(&u, &f, sizeof(float));     // convert float to bit pattern
        sc_lv<34> b = 0;
        b.range(33,32) = (i+1 == p->datas.size()) ? "01" : "00";  // tail or body
        b.range(31,0) = sc_lv<32>(u);       // data bits
        v.push_back(b);
    }
    return v;
}
```

- flits2packet( ): 將 flits 還原成 packet

```cpp
Packet* Core::flits2packet(const std::vector<sc_lv<34> >& v) {
    Packet* p = new Packet;
    sc_lv<34> h = v[0];
    p->source_id = h.range(31,16).to_uint();
    p->dest_id   = h.range(15,0).to_uint();
    for (size_t i = 1; i < v.size(); ++i) {
        sc_lv<32> d = v[i].range(31,0);
        unsigned u = d.to_uint();
        float f;
        std::memcpy(&f, &u, sizeof(float));  // convert bits to float
        p->datas.push_back(f);
    }
    return p;
}
```

2. Router Design (router.h)

功能：

- receive、buffer 與 routing flits
- 根據 packet 的目的地（header flit）進行 routing 算法
- 實作 XY routing 演算法

重要變數：

- in_flit / out_flit：傳輸資料（sc_lv<34>）
- in_req / out_req：表示資料傳輸請求
- in_ack / out_ack：確認傳輸成功
- stored_o[5]: 紀錄每個輸入 port 的目標輸出 port
- locked_by[5]: 紀錄每個輸出 port 被哪一個輸入 port 鎖住（避免 flit 混亂）
- in_buf[5] / out_buf[5]: 各方向的輸入與輸出 buffer (使用 deque 暫存 flit 資料，實現非阻塞 dequeue)

```
    sc_out < sc_lv<34> >  out_flit[5];
    sc_out < bool >  out_req[5];
    sc_in  < bool >  in_ack[5];

    sc_in  < sc_lv<34> >  in_flit[5];
    sc_in  < bool >  in_req[5];
    sc_out < bool >  out_ack[5];

    int router_id;      // router id
    int stored_o[5];    // store selected output direction for each input
    int locked_by[5];   // output port lock owner (-1 if free)
    std::deque<sc_lv<34> > in_buf[5];
    std::deque<sc_lv<34> > out_buf[5];
```

Routing 片段: 若接收到 header flit 時，呼叫 compute_dir() 決定下一跳方向，並設置方向鎖避免衝突。若接收到 tail flit 則先存至 buffer 裡面並解鎖方向鎖

```
if (stored_o[i] == -1) {
    // header
    if (type == "10") {
        int dst = f.range(15,0).to_uint();  // get destination
        int o = compute_dir(dst);   // compute destination
        if (locked_by[o] == -1) {
            stored_o[i] = o;
            locked_by[o] = i;
        }
    }
}

int o = stored_o[i];
if (locked_by[o] == i && out_buf[o].size() < 4) {
    out_buf[o].push_back(f);
    in_buf[i].pop_front();
    // tail
    if (type == "01") {
        stored_o[i] = -1;
        locked_by[o] = -1;
    }
}
```

Write 片段: 當該輸出口 buffer 有資料且對方 ack 時，送出 flit 並將其移出緩衝

```
// transmit flit
for (int o = 0; o < 5; ++o) {
    if (!out_buf[o].empty()) {
        out_flit[o].write(out_buf[o].front());
        out_req[o].write(true);

        if (in_ack[o].read()) {
            out_buf[o].pop_front();
        }
    }
    else {
        out_req[o].write(false);
    }
}
```

3. Main (main.cpp)

功能：

- 建立 Clock 與 Reset
- 初始化 16 組 Core 與 Router
- 宣告所有信號（sc_signal）
- 連接：
  - Core ⇌ Router 的 LOCAL port
  - Router ⇌ Router 四個方向的 port（E/S/W/N）
  - 邊界 Router 的 dummy port（無鄰居）

LOCAL port 連接函式:

```
// local port connections between cores and routers
void connect_local(Core* core, Router* router, sc_signal<sc_lv<34> > (&flit)[2], sc_signal<bool> (&req)[2], sc_signal<bool> (&ack)[2]) {
    // core (tx) -> router
    core->flit_tx(flit[0]);              // sc_out<sc_lv<34> > flit_tx;
    router->in_flit[LOCAL](flit[0]);     // sc_in<sc_lv<34> > in_flit[5];
    core->req_tx(req[0]);               // sc_out<bool> req_tx;
    router->in_req[LOCAL](req[0]);      // sc_in<bool> in_req[5];
    router->out_ack[LOCAL](ack[1]);     // sc_out<bool> out_ack[5];
    core->ack_tx(ack[1]);               // sc_in<bool> ack_tx;

    // router -> core (rx)
    router->out_flit[LOCAL](flit[1]);   // sc_out<sc_lv<34> > out_flit[5];
    core->flit_rx(flit[1]);             // sc_in<sc_lv<34> > flit_rx;
    router->out_req[LOCAL](req[1]);     // sc_out<bool> out_req[5];
    core->req_rx(req[1]);               // sc_in<bool> req_rx;
    core->ack_rx(ack[0]);               // sc_out<bool> ack_rx;
    router->in_ack[LOCAL](ack[0]);      // sc_in<bool> in_ack[5];
}
```

四個方向連接函式:

```
// router to router connections based on directions
void connect_dir(Router* from, Router* to, Dir out_dir, Dir in_dir, sc_signal<sc_lv<34> >& flit, sc_signal<bool>& req, sc_signal<bool>& ack) {
    // send packet
    from->out_flit[out_dir](flit);
    from->out_req[out_dir](req);
    from->in_ack[out_dir](ack);

    // receive packet
    to->in_flit[in_dir](flit);
    to->in_req[in_dir](req);
    to->out_ack[in_dir](ack);
}
```

邊界連接函式:

```
// edge connections for the last row and column (assuming edge connections are similar to router connections, but not connected to other routers)
void connect_edge(Router* r, Dir dir, sc_signal<sc_lv<34> >& flit1, sc_signal<bool>& req1, sc_signal<bool>& ack1, sc_signal<sc_lv<34> >& flit2,
                  sc_signal<bool>& req2, sc_signal<bool>& ack2) {
    r->out_flit[dir](flit1);
    r->out_req[dir](req1);
    r->in_ack[dir](ack1);
    r->in_flit[dir](flit2);
    r->in_req[dir](req2);
    r->out_ack[dir](ack2);
}
```

三、 Routing 演算法

使用「XY Routing」（先水平再垂直）算法：

- 若目標位置在右側，則先往東，左側則往西
- 當水平位置抵達後，再往南或北

函式 compute_dir() 依據當前 router id 與 pe id 計算方向：

```
// calculate destination (route in x direction first, then y)
int Router::compute_dir(int dest) {
    int x  = router_id % 4;
    int y = router_id / 4;
    int dx = dest % 4;
    int dy = dest / 4;

    if (x < dx) return EAST;
    if (x > dx) return WEST;
    if (y < dy) return SOUTH;
    if (y > dy) return NORTH;
    return LOCAL;
}
```

四、 遇到的挑戰與解決方案

一開始最麻煩的是 Router 的方向連線，因為每個 Router 都有東、南、西、北、Local 五個方向可以接，還要互相連到其他 Router。如果寫死的話很容易接錯，我後來就自己寫了 connect_dir() 和 connect_local() 這種小函式，讓連線比較直覺，也比較不容易出錯。

再來是 flit 的格式，我一開始就搞錯了。原本沒有仔細定義好每個 bit 的意思，導致 packet 送過去之後 PE 無法驗證通過，一直爆錯。後來我改成這樣：

- [33:32] 是 flit 的類型（10 : header，00 : body，01 : tail）
- [31:16] 是 source id
- [15:0] 是 destination id

這樣之後 flit 在 router 裡就可以正確 routing，也能在 core 那邊重組回原本的 packet。

五、 心得

這次作業老實說真的不簡單，但我學到很多。以前在課本或簡報上看到 NoC 就只覺得它是個很抽象的東西，說是用來讓多個處理器互相傳資料，但沒看過實際運作。這次是我第一次真的把它從零寫出來，而且每個 flit 的流向、每個 router 的轉向、每個 buffer 的資料，都能一步步慢慢 trace 出來。雖然過程中遇到不少 bug，但 debug 成功後真的滿有成就感。

而寫完程式後我也更了解什麼是 NoC。以前覺得它只是個名詞，現在知道其實它就像是一個片上的小網路，裡面的 Router 就像 switch，Core 像一台台主機，資料要一站站傳過去，不能亂走，也不能走錯。

這次我最有感的一點，是在 SystemC 裡思考時間跟狀態的概念。不是像一般寫 C++ 那樣一行一行執行，而是你要想像這是一個會隨 clock 前進的世界，每個模組都是平行運作的。你要考慮的不是只是 if-else，而是「在這個時間點這個訊號該不該變？對面收到了沒？下一個 flit 要不要等？」這些觀念都是做這個作業才真正體會到的。

這次作業雖然很花時間，但也學到了很多。最重要的是可以從不了解 flit 是什麼，到一步步建立出一個完整、可運作的 NoC 系統。這種從 0 到 1 的過程雖然痛苦，但很值得。